

TARGERIAN

VS.

BARATHEON

INFORME FINAL DEL PROYECTO

Videojuego 2D desarrollado con Qt 6 / C++

Participantes

Nombre 1: isaac garcia quiros

Nombre 2: sebastian bedoya

Informatica2 | Proyecto final

2026

Qt 6 • C++ • Programación Orientada a Objetos

Índice

1. Introducción	2
2. Planteamiento del Problema	2
3. Objetivos	2
4. Requerimientos	3
4.1. Funcionales	3
4.2. No funcionales	3
4.3. Restricciones	3
5. Metodología y Planificación	3
6. Diseño y Arquitectura	4
6.1. Jerarquía de clases	4
7. Implementación	5
7.1. Animaciones por sprite sheet	5
7.2. Inteligencia artificial adaptativa	5
7.3. Sistema de físicas de empuje	5
7.4. Detección de colisiones	5
8. Pruebas	5
9. Instalación y Controles	6
9.1. Requisitos	6
9.2. Compilación	6
9.3. Controles	6
10.Resultados y Discusión	7
11.Conclusiones	7
12.Referencias	7
A. Estructura de archivos	8
B. Fragmentos de código relevantes	8

1. Introducción

Targeryan vs. Baratheon es un videojuego 2D de acción desarrollado en C++ con el framework Qt 6, inspirado en el universo de *Canción de Hielo y Fuego*. El proyecto integra programación orientada a objetos, inteligencia artificial básica y manejo de multimedia en un producto interactivo completo.

El juego tiene dos niveles: un **runner de esquivas** con dificultad escalable y un **combate 1v1** entre Robert Baratheon (jugador) y Rhaegar Targeryan (IA).

2. Planteamiento del Problema

Los cursos de programación avanzada requieren proyectos que demuestren la interacción real entre subsistemas: lógica de juego, inteligencia artificial, gráficos y multimedia. Este proyecto responde a esa necesidad construyendo un videojuego completo que cubre:

- Diseño orientado a objetos con herencia y polimorfismo.
- Manejo de estados y transiciones entre niveles.
- Agente de IA con comportamiento adaptable.
- Integración de sprites, audio y video.
- Ciclo de actualización en tiempo real (60 FPS).

3. Objetivos

Objetivo general

Desarrollar un videojuego 2D funcional con dos niveles diferenciados, personajes animados e IA adaptativa, usando Qt 6 y C++.

Objetivos específicos

- Implementar una jerarquía de clases extensible (Entidad, Jugador, Obstáculo, Nivel).
- Crear una IA para Rhaegar con comportamiento adaptativo según el estado del combate.
- Animar personajes mediante *sprite sheets* con soporte de espejado.
- Diseñar un HUD que muestre vida y escudo en tiempo real.
- Integrar audio (efectos/música) y video (cutscenes) con QMediaPlayer.

4. Requerimientos

4.1 Funcionales

ID	Descripción	Módulo
RF-01	Menú principal con opciones Normal y Difícil.	MainWindow
RF-02	Nivel 1: obstáculos en carriles con desplazamiento vertical.	Nivel1
RF-03	Nivel 1: jefe final a los 50s, bloquea al jugador en carril central.	Nivel1
RF-04	Nivel 2: ataque (Z), bloqueo (X), movimiento (A/D) y salto (W).	Nivel2
RF-05	IA adapta agresividad según vida del jugador y patrón de combate.	IA
RF-06	Escudo de ambos personajes se regenera periódicamente.	Nivel2
RF-07	Oleadas de flechas periódicas que dañan a ambos jugadores.	Nivel2
RF-08	Cutscene de video entre niveles.	MainWindow
RF-09	Pantallas de victoria y derrota al finalizar el Nivel 2.	Juego

4.2 No funcionales

- **Rendimiento:** lógica a 60 FPS (timer de 16 ms).
- **Portabilidad:** compatible con Windows, Linux y macOS (Qt 6).
- **Mantenibilidad:** niveles desacoplados mediante interfaz abstracta `Nivel`.
- **Escalabilidad:** nuevos niveles sin modificar la clase `Juego`.

4.3 Restricciones

- Sin motor de físicas externo: gravedad y empujes calculados manualmente.
- Modo un jugador vs. IA (sin multijugador en red).
- Recursos gráficos y de audio embebidos en el sistema QRC de Qt.

5. Metodología y Planificación

El desarrollo siguió un modelo iterativo e incremental con entregas semanales:

Fase	Actividades	Duración
Análisis	Definición de clases y requisitos	1 semana
Diseño	Diagramas y arquitectura de niveles	1 semana
Implementación N1	Runner, obstáculos, jefe, dificultad	2 semanas
Implementación N2	Combate, sprites, IA, flechas, escudo	3 semanas
Multimedia	Audio, video, cutscenes, menú	1 semana
Pruebas	Verificación manual y ajuste de balance	1 semana
Total		9 semanas

6. Diseño y Arquitectura

6.1 Jerarquía de clases

El sistema se organiza en cuatro capas:

- **Entidades:** Entidad (base abstracta) → Jugador, Obstáculo.
- **Niveles:** Nivel (interfaz) → Nivel1, Nivel2.
- **Lógica:** Juego (orquestador), IA (agente inteligente).
- **Presentación:** MainWindow (renderizado, eventos, audio/video).

Clase	Responsabilidad
Entidad	Base de todos los objetos: posición, hitbox y estado activo.
Jugador	Movimiento, salto, gravedad, animaciones, vida, escudo y entrada de teclado.
Obstáculo	Bloques del Nivel 1 y flechas del Nivel 2 con física de proyectil.
Nivel	Interfaz abstracta: iniciar, actualizar, renderizar, terminado.
Nivel1	Runner con obstáculos por patrones, dos dificultades y jefe final.
Nivel2	Arena de combate con físicas, regeneración de escudo e IA.
IA	Decisiones cada 160 ms según distancia, vida y agresividad.
Juego	Ciclo de vida: crea, actualiza y destruye niveles.
MainWindow	Timer de 60 FPS, eventos de teclado, QPainter, audio y video.

7. Implementación

7.1 Animaciones por sprite sheet

Cada sprite sheet se divide en filas (estados: caminar, atacar, bloquear, recibir golpe) y columnas (frames). Rhaegar usa una cuadrícula de 7×4 (celda 219×243 px); Robert, 8×4 (celda 192×256 px). El espejado horizontal se precalcula con `QTransform().scale(-1, 1)`, almacenando arrays paralelos para evitar transformaciones en tiempo de ejecución.

7.2 Inteligencia artificial adaptativa

La IA evalúa el estado del combate cada 160 ms. Sus prioridades de decisión son:

1. Vida del jugador muy baja (<20 HP): máxima presión y ataques con *cooldown* reducido.
2. Vida del jugador baja (<40 HP): modo presión con saltos frecuentes.
3. Modo presión activo: perseguir y atacar sin condiciones adicionales.
4. Jugador bloqueando: mantener distancia segura.
5. Escudo de la IA agotado: retroceder y recuperar.

La **agresividad** es un valor entre 0.20 y 1.0 que sube al recibir daño y baja al fallar ataques, generando un comportamiento emergente que varía en cada partida.

7.3 Sistema de físicas de empuje

Los golpes aplican impulsos proporcionales a la masa de cada personaje (`MASA_ROBERT = 2`, `MASA_RHAEGAR = 1`). La velocidad de empuje decae por fricción cada frame y se cancela al bajar de 8 px/s.

7.4 Detección de colisiones

Se usan rectángulos AABB (*Axis-Aligned Bounding Boxes*). En el Nivel 1 se aplica `QRect::intersects()`. En el Nivel 2, las hitboxes de ataque se expanden dinámicamente hacia el lado del defensor con un rango adicional de 30 px.

8. Pruebas

Las pruebas fueron manuales funcionales y de integración. La tecla I permite activar/desactivar la IA en el Nivel 2 para verificar el comportamiento del jugador de forma aislada.

Caso de prueba	Resultado esperado	Resultado
Colisión con obstáculo N1	Incrementa choques, sonido de daño	Correcto
10 choques → fin de Nivel 1	Transición inmediata al Nivel 2	Correcto
Jefe final a los 50 s	Jugador en centro, jefe visible	Correcto
Ataque con escudo activo (N2)	Reduce escudo, no la vida	Correcto
IA con vida crítica (<20 HP)	Mayor velocidad y frecuencia de ataque	Correcto
Flecha impacta sin escudo	Reduce vida directamente	Correcto
Salto sobre flechas	Hitbox elevada evita colisión	Parcial
Victoria/derrota detectada	Pantalla correcta según resultado	Correcto

9. Instalación y Controles

9.1 Requisitos

Qt 6.4+ (Widgets, Multimedia, MultimediaWidgets), compilador MSVC 2022/GCC 11+/Clang 14+, CMake 3.21+ y al menos 512 MB de RAM.

9.2 Compilación

Abrir el proyecto en Qt Creator, cargar `CMakeLists.txt` o el archivo `.pro`, seleccionar configuración *Release* y compilar con `Ctrl+B`.

9.3 Controles

Tecla	Acción
A / D	Mover izquierda / derecha
W	Saltar (Nivel 2)
Z	Atacar (Nivel 2)
X	Bloquear — mantener (Nivel 2)
I	Activar/desactivar IA (modo debug)

10. Resultados y Discusión

El juego cumple todos los requisitos funcionales. El Nivel 1 ofrece una experiencia fluida con dificultad perceptible; el Nivel 2 presenta un combate dinámico donde la IA representa un reto real cuando su agresividad supera 0.70.

Limitaciones:

- La IA no esquiva flechas activamente.
- No hay sistema de puntuación ni tabla de clasificación.
- Las hitboxes AABB del Nivel 2 son menos precisas que las del sprite real.

Mejoras planeadas:

- Tercer nivel con mecánicas de plataformas.
- Soporte para gamepad (QGamepad).
- Reemplazar la IA reactiva por un árbol de comportamiento (*Behavior Tree*).

11. Conclusiones

El proyecto demostró que es posible construir un videojuego 2D completo usando exclusivamente Qt 6 y C++ estándar. Los aprendizajes clave fueron:

- La interfaz abstracta `Nivel` facilitó agregar el Nivel 2 sin modificar el orquestador Juego.
- La IA adaptativa genera comportamientos emergentes que hacen cada partida diferente.
- El *balance* de juego consume más tiempo del esperado; conviene establecer métricas desde el inicio.

12. Referencias

1. Qt Group. *Qt 6 Documentation*. <https://doc.qt.io/qt-6/>
2. Millington, I. & Funge, J. (2009). *Artificial Intelligence for Games* (2nd ed.). Morgan Kaufmann.
3. Nystrom, R. (2014). *Game Programming Patterns*. <https://gameprogrammingpatterns.com/>
4. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
5. Martin, R. C. (2008). *Clean Code*. Prentice Hall.

A. Estructura de archivos

Archivo	Contenido
main.cpp	Punto de entrada. Crea QApplication y MainWindow.
mainwindow.h/.cpp	Ventana principal, bucle, renderizado, audio y video.
juego.h/.cpp	Orquestador de niveles y detección de fin de partida.
nivel.h	Interfaz abstracta para todos los niveles.
nivel1.h/.cpp	Runner con obstáculos, jefe y dificultad.
nivel2.h/.cpp	Arena de combate con IA, flechas y empuje.
jugador.h/.cpp	Personaje: animaciones, movimiento y combate.
entidad.h/.cpp	Base abstracta: posición, hitbox y estado activo.
ia.h/.cpp	Agente IA para Rhaegar con comportamiento adaptativo.
obstaculo.h/.cpp	Obstáculos del Nivel 1 y flechas del Nivel 2.

B. Fragmentos de código relevantes

Reparto de masa en empujes (nivel2.cpp)

```
float masaTotal = MASA_ROBERT + MASA_RHAEGAR; // 3
velEmpuje2 += dir * FUERZA_EMPUJE * (MASA_ROBERT / masaTotal); // 2/3
velEmpuje1 -= dir * FUERZA_EMPUJE * (MASA_RHAEGAR / masaTotal); // 1/3
```

Adaptación de agresividad (ia.cpp)

```
// Al recibir un golpe:
agresividad += 0.05f; if (agresividad > 1.0f) agresividad = 1.0f;
// Al fallar un ataque:
agresividad -= 0.02f; if (agresividad < 0.20f) agresividad = 0.20f;
```

Intervalo de regeneración de escudo (nivel2.cpp)

```
return INTERVALO_ESCUDO_BASE + (1.0f - proporcionVida)
    * INTERVALO_ESCUDO_EXTRA;
```